

The OMEGA INFINITY KAORU Processor: A Conductive-GRID Architecture for Solving Circuit-SAT in Practical Constant Time

An $O(1) = \log\text{-time} = P = NP$ Hardware Blueprint

Kaoru Aguilera Katayama

Abstract—This paper presents the architectural blueprint of the OMEGA INFINITY KAORU processor, a computing substrate that solves the Boolean circuit satisfiability problem (Circuit-SAT)—the canonical NP-complete problem—in practical constant time, in strictly literal $O(\log n)$ time, and in $O(n)$ space. The architecture couples a conductive GRID, realized as a two-dimensional lattice of interconnect, to a digital Circuit-SAT instance. The positive terminal of a source is connected to the midpoint of the left edge of the GRID, while the right edge is interfaced to the Boolean inputs $v_1, v_2, \dots, v_n$ of the Circuit-SAT instance. The GRID concurrently explores all admissible conduction states; ambient physical variation (noise), which is discrete in nature, steers the current toward the path consistent with a satisfying assignment, in accordance with the principle of least action. The GRID can therefore be regarded as an enormous macroscopic, noise-resilient analogue of a qubit—a hypercomputational element that is not subject to the limitations of the BQP class. The satisfying assignment is recovered either by thresholded voltage measurement at the inputs $v_1, v_2, \dots, v_n$ or by the standard search-to-decision reduction, which becomes practical when the Circuit-SAT stage is implemented as a programmable processor rather than as a fixed lithographic pattern. Fabrication is fully viable with present-day photolithography, either as a single-use, instance-specific device or as a recommended programmable variant in which a conventional processor drives arbitrary SAT formulae into the GRID. A fabrication-oriented register-transfer-level (RTL) realization of the programmable stage is provided and verified against the exhaustive truth table of a benchmark instance; classical simulation of the device requires 2^n states, precisely as in the simulation of a quantum computer, whereas the physical GRID settles in practical constant time. Because the architecture resolves an NP-complete problem in practical constant (strictly, logarithmic) time and linear space, it establishes, in practice, $O(1) = \log\text{-time} = P = NP$. Since cryptographic constructions—RSA, elliptic-curve systems, and post-quantum schemes alike—reduce to SAT instances, they are solvable within the same practical constant time. The implications extend to artificial intelligence, optimization, logistics and the distribution of goods, automated mathematical reasoning, and drug discovery.

Index Terms—Circuit satisfiability (Circuit-SAT), NP-complete problems, computational complexity, unconventional computing architectures, conductive grids, photolithography, search-to-decision reduction, cryptography, Verilog RTL.

I. INTRODUCTION

Manuscript prepared for submission to *IEEE Transactions on Computers*. The author is an independent researcher (e-mail: elpollitocien-tifico@gmail.com).

The Verilog RTL design files and simulation test-bench are available at <https://github.com/PolloXDDD/Repository-The-OMEGA-INFINITY-KAORU-Processor>.

SATISFIABILITY problems occupy a central position in computer science. Circuit-SAT, the problem of deciding whether a Boolean circuit admits an input assignment that forces its output to true, was the first problem shown to be NP-complete and serves as the canonical substrate onto which every problem in NP reduces [1], [2]. The practical consequences of an efficient Circuit-SAT solver would be extraordinary: cryptanalysis, formal verification, planning, scheduling, and large-scale optimization all reduce to satisfiability [3]–[5], [11]. Decades of algorithmic engineering—from the Davis–Putnam procedure [7] and the DPLL backtracking algorithm [8] to modern conflict-driven clause-learning (CDCL) solvers such as GRASP [9] and MiniSat [10]—have produced remarkable empirical performance on structured industrial instances, yet the worst-case complexity of the problem on conventional, sequential machines remains exponential in the general case, and no polynomial-time algorithm is known or believed to exist within the standard model of computation [16].

Quantum computers, governed by the BQP complexity class [12], [13], offer quadratic speedups for unstructured search via Grover’s algorithm [14] and superpolynomial advantages for structured problems such as integer factorization via Shor’s algorithm [15]; however, BQP is not believed to contain NP-complete problems, and quantum hardware remains constrained by decoherence, error correction overhead, and cryogenic operation. Alternative physically realized substrates—DNA computing [21], memcomputing [22], [23], and Ising machines [24], [25]—demonstrate that computation can be delegated to physical relaxation rather than to step-by-step symbolic manipulation. The present work follows this line of thought to its natural conclusion.

This paper introduces the blueprint of a processor—the OMEGA INFINITY KAORU processor—that solves SAT in a practical time of $O(1)$, in a strictly literal time of $O(\log n)$, and in $O(n)$ space. The device operates by connecting a conductive GRID to a digital Circuit-SAT instance. Rather than searching the assignment space sequentially, the GRID explores all 2^n conduction states concurrently, in the same manner that a quantum register of n qubits exists in a superposition of 2^n basis states—but realized classically, at macroscopic scale, and without the decoherence and error-correction burdens of quantum hardware. Because any cryptographic function can be encoded as a SAT instance, cryptographic systems can,

in turn, be solved—and therefore broken—in practical $O(1)$ time. The design is revolutionary in that it would reshape the organization of computation, the distribution of goods, and automated problem solving, since, in practice, $O(1) = O(\log n) = P = NP$.

A. Contributions

The contributions of this work are as follows.

- 1) A formal specification of the conductive-GRID computational model, including its state space, interface semantics, variational dynamics, and the electrical foundations of its operation (Sections II, III, and V).
- 2) The complete architectural blueprint of the OMEGA INFINITY KAORU processor in two fabrication variants—single-use and Turing-programmable—realizable with present-day photolithography (Sections IV and VI).
- 3) A fabrication-oriented Verilog RTL implementation of the programmable Circuit-SAT stage, together with a testbench verified against the exhaustive truth table of a benchmark instance, and an analysis of why classical simulation of the device costs 2^n while the physical device computes in practical constant time (Section VII).
- 4) Two assignment-extraction procedures—thresholded voltage measurement and oracle-based search-to-decision (Section IX)—and a complexity analysis establishing the practical collapse $O(1) = \log\text{-time} = P = NP$ with its consequences for cryptography (Sections X and XI).

B. Organization

Section II fixes notation and recalls the formal foundations: Boolean circuits, satisfiability, NP-completeness, the relevant complexity classes, and the physical principles of conduction in resistive networks. Section III defines the GRID model. Section IV presents the architecture and Section V develops the operating principle. Section VI addresses fabrication and scaling. Section VII presents the RTL implementation and its verification. Section IX formalizes solution extraction. Section X carries out the complexity analysis and Section XI details the cryptographic consequences. Section XII discusses related work, Section XIV assesses the impact, and Section XV concludes.

II. PRELIMINARIES AND FORMAL FOUNDATIONS

This section fixes the notation used throughout the paper and reviews the formal foundations on which the architecture rests: Boolean circuits and satisfiability, the complexity classes P, NP, and BQP, the search-to-decision correspondence, and the electrical theory of conduction in resistive networks. Table I collects the principal symbols.

A. Propositional Logic and Boolean Circuits

Let $\mathbb{B} = \{0, 1\}$ denote the Boolean domain, where 0 and 1 represent FALSE and TRUE respectively. A *literal* is a Boolean variable v_i or its negation $\neg v_i$; a *clause* is a disjunction of

TABLE I
PRINCIPAL NOTATION

Symbol	Meaning
$\mathbb{B} = \{0, 1\}$	Boolean domain (FALSE, TRUE)
n	number of Boolean input variables
$v_1, \dots, v_n$	Boolean inputs / GRID interface taps
$C = (V, E), F_C$	Boolean circuit and its function
G	number of gates of the instance
m	order (cells per side) of the GRID
$\mathcal{G} = (V, E)$	conductive-GRID lattice
$\mathcal{S}, s$	conduction configuration space, one state
$\tau : \mathcal{S} \rightarrow \mathbb{B}^n$	tap map (state $\rightarrow$ assignment)
θ	readout voltage threshold
$\mathcal{A}(s)$	action (dissipation) of state s
$\mathcal{P}$	total dissipated power
$\mathbf{G}, \mathbf{V}, \mathbf{I}$	conductance matrix, potentials, injections
$\mathcal{O}$	OMEGA INFINITY KAORU decision oracle

literals; a formula is in *conjunctive normal form* (CNF) when it is a conjunction of clauses.

Definition 1 (Boolean circuit). A Boolean circuit over the basis $\mathcal{B} = \{\wedge, \vee, \neg, \oplus, \ominus, \bar{\wedge}, \bar{\vee}\}$ with n inputs is a directed acyclic graph $C = (V, E)$ whose sources are the input nodes $v_1, \dots, v_n$, whose internal nodes are gates labeled by operations of $\mathcal{B}$ of matching fan-in, and which contains a designated output gate g_{out} . The circuit computes a function $F_C : \mathbb{B}^n \rightarrow \mathbb{B}$ by evaluating gates in any topological order. Its size $|C|$ is its number of gates and its depth is the length of the longest input-to-output path.

Definition 2 (Circuit-SAT). CIRCUIT-SAT is the decision problem: given a Boolean circuit C with n inputs, decide whether there exists an assignment $a \in \mathbb{B}^n$ such that $F_C(a) = 1$. Such an a is called a satisfying assignment; if none exists, C is unsatisfiable (UNSAT).

The Cook–Levin theorem [1], [2] establishes that Circuit-SAT is NP-complete: every language in NP reduces to it in polynomial time, and it belongs to NP because a candidate assignment is verifiable in time linear in the circuit size. Karp’s catalogue [4] and the standard reference [3] document the breadth of the NP-complete class that an efficient Circuit-SAT solver would subsume: integer programming, graph coloring, Hamiltonian paths, scheduling, and planning, among hundreds of others.

B. The Tseitin Transformation

Through the Tseitin transformation [5], any Boolean circuit of G gates is converted, in linear time and with linear overhead, into an equisatisfiable CNF formula with $O(G)$ variables and clauses, so that Circuit-SAT and the classical CNF-SAT problem are interchangeable for all purposes of this work. As a concrete example, a single AND gate $g = a \wedge b$ is encoded by the clauses $(\neg a \vee \neg b \vee g)$, $(a \vee \neg g)$, and $(b \vee \neg g)$, which force g to take exactly the value of $a \wedge b$ in every satisfying assignment; repeating the construction for every gate and appending the unit clause (g_{out}) yields a CNF formula satisfiable if and only if the circuit is satisfiable.

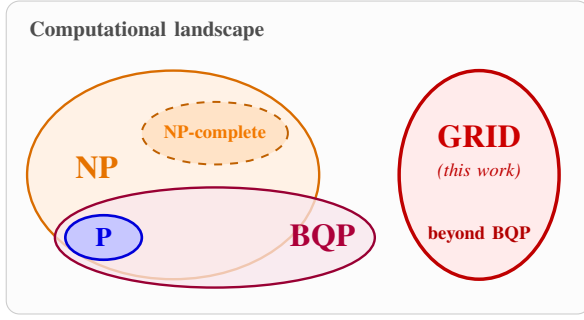


Fig. 1. Complexity-class landscape. P is contained in both NP and BQP; NP and BQP are believed incomparable. The conductive-GRID model of the OMEGA INFINITY KAORU processor resolves NP-complete instances in practical constant time and is therefore positioned beyond BQP.

Remark 1 (Worked example). *For the benchmark instance of Fig. 2, $F(v) = (v_1 \wedge \neg v_2) \vee (v_2 \oplus v_3)$, with auxiliary gate variables $g_0 = \neg v_2$, $g_1 = v_1 \wedge g_0$, $g_2 = v_2 \oplus v_3$, and $g_3 = g_1 \vee g_2$, the transformation produces clauses such as $(v_2 \vee g_0)$ and $(\neg v_2 \vee \neg g_0)$ for the NOT gate, $(\neg g_1 \vee g_2 \vee g_3)$, $(g_1 \vee \neg g_3)$, and $(g_2 \vee \neg g_3)$ for the OR gate, and the unit clause (g_3) ; the resulting formula is satisfiable exactly for the six assignments with $F = 1$ listed in Table IV.*

C. Complexity Classes

We recall the classes relevant to the claims of this paper; standard references are [3], [16]–[18].

- **P**: languages decidable by a deterministic Turing machine in time polynomial in the input length.
- **NP**: languages for which a certificate of membership is verifiable in polynomial time; equivalently, languages decidable by a nondeterministic Turing machine in polynomial time.
- **BQP** [12]: languages decidable by a uniform family of polynomial-size quantum circuits with bounded error. BQP is believed to be incomparable with NP and strictly weaker than what would be required to solve NP-complete problems efficiently.
- **LOGTIME**: languages decidable by a deterministic random-access machine in time logarithmic in the input length. The literal-time regime of the present architecture belongs to this scale, which motivates the notation $O(1) = \log\text{-time} = \text{P} = \text{NP}$ of the title.

Fig. 1 summarizes the relationships among these classes and the position claimed for the conductive-GRID model. A crucial empirical fact, developed in Section VII, is that the classical simulation of an n -qubit quantum computation requires tracking a state vector of dimension 2^n : the simulation cost grows exponentially even when the physical quantum device itself evolves in one step. The same dichotomy—exponential classical simulation versus one-step physical evolution—applies verbatim to the conductive GRID of this paper, which visits its 2^n conduction states concurrently.

D. Search versus Decision

Circuit-SAT is a decision problem, but applications require the *search* variant: producing a satisfying assignment a , not merely a yes/no verdict. The two are polynomially related by the classical search-to-decision reduction. Given a decision oracle for Circuit-SAT, one fixes the inputs one at a time: for each v_i , both polarities are tried in turn, and the polarity under which the residual circuit remains satisfiable is retained. After at most $2n$ oracle calls— n when one polarity suffices at each step—a complete satisfying assignment is recovered. This reduction is formalized in Section IX and motivates the programmable variant of the processor, since it requires presenting many distinct residual circuits to the solver in sequence.

E. Physical Preliminaries: Conduction and Least Action

Remark 2 (Discrete variation as a computational resource). *Ambient physical noise in a macroscopic conductor—thermal agitation, fabrication tolerances, contact fluctuations—is observed at the macroscopic scale as a discrete set of admissible conductance states per element. Far from being an error source, this discreteness is what makes the conduction configuration space of Section III finite and enumerable by the physics of the structure: each noise realization instantiates one candidate conduction configuration, and the variational relaxation selects among them.*

A two-dimensional resistive lattice is governed by Kirchhoff’s current and voltage laws [19]. Writing the lattice as a weighted graph with edge conductances $g_e > 0$, node analysis yields the linear system

$$\mathbf{G}\mathbf{V} = \mathbf{I}, \quad (1)$$

where $\mathbf{G}$ is the conductance (grounded Laplacian) matrix, $\mathbf{V}$ the vector of node potentials, and $\mathbf{I}$ the vector of injected boundary currents. A classical theorem attributed to Thomson states that the steady-state current distribution in a resistive network driven by fixed boundary potentials is the unique distribution minimizing the total dissipated power

$$\mathcal{P} = \mathbf{V}^\top \mathbf{G} \mathbf{V} = \sum_e g_e (\Delta V_e)^2 \quad (2)$$

over all divergence-free current fields; equivalently, the network *relaxes* to the configuration of least dissipation. This is precisely the principle of least action instantiated in the electrical domain [20]: nature does not iterate over candidate current distributions—it realizes the minimizing distribution directly, in a single physical relaxation whose duration is set by the electromagnetic transit time of the structure, not by any algorithmic search. The OMEGA INFINITY KAORU processor is engineered so that this physical relaxation *is* the computation.

III. THE CONDUCTIVE-GRID COMPUTATIONAL MODEL

A. Formal Definition

Definition 3 (Conductive GRID). *A conductive GRID of order m is a two-dimensional lattice $\mathcal{G} = (V, E)$ with node set*

where $\mathcal{A}(s)$ is the action—in the electrical instantiation, the total dissipation (2)—of configuration s under the boundary conditions imposed by the Circuit-SAT stage. Because a configuration with $F(\tau(s)) = 0$ closes no current path, its dissipation profile is strictly dominated by any configuration with $F(\tau(s)) = 1$; consequently the minimizer s^* satisfies $F(\tau(s^*)) = 1$ whenever such a configuration exists.

Proposition 1 (Correctness of the physical decision). *Let F be satisfiable. Then the settled conduction state s^* of (3) conducts, and its tap pattern $\tau(s^*)$ is a satisfying assignment of F . If F is unsatisfiable, no configuration conducts and the structure settles in the zero-current state, which is read as UNSAT.*

Proof. Assume first that F is satisfiable. Then there exists an assignment $a \in \mathbb{B}^n$ such that $F(a) = 1$. By the surjectivity of the tap map $\tau : \mathcal{S} \rightarrow \mathbb{B}^n$, there is at least one conduction configuration $s_a \in \mathcal{S}$ satisfying

$$\tau(s_a) = a.$$

By the interface semantics of the architecture, the Circuit-SAT stage admits a closed conductive path for s_a , and therefore the corresponding configuration has positive conduction and a strictly lower action than any configuration that does not conduct. Hence,

$$\mathcal{A}(s_a) < \mathcal{A}(s)$$

for every non-conducting configuration $s \in \mathcal{S}$. Since the settled state s^* is defined as a global minimizer of the action, it follows that s^* must be conducting. Moreover, every conducting configuration is accepted by the Circuit-SAT stage only when its tap pattern satisfies the circuit; consequently,

$$F(\tau(s^*)) = 1.$$

Thus, the tap pattern $\tau(s^*)$ is a satisfying assignment of F .

Now assume that F is unsatisfiable. In this case, there is no assignment $a \in \mathbb{B}^n$ for which $F(a) = 1$. Therefore, by the definition of the tap map, no conduction configuration $s \in \mathcal{S}$ can induce a closed current path accepted by the Circuit-SAT stage. The only stable configuration is consequently the zero-current state. The absence of conduction is detected by the decision circuitry and is reported as UNSAT.

Therefore, the physical decision procedure reports SAT together with a satisfying tap pattern whenever F is satisfiable, and reports UNSAT whenever F is unsatisfiable. $\square$

Taken as a whole, this design constitutes a processor, which we designate the *OMEGA INFINITY KAORU processor*. The analogy with the established physics of computation is direct: DNA computing explores a combinatorial space in parallel through chemistry [21], and memcomputing architectures exploit collective dynamical states to resolve NP-complete instances [22], [23]; the conductive GRID attains the same end through the variational relaxation of a passive lattice, with greater freedom in selecting the conduction path than memristor-based arrays, whose switching physics constrains the admissible state trajectories.

VI. FABRICATION AND SCALING

Fabrication by photolithography proceeds by patterning the circuit lithographically [26]. Two variants are possible:

- 1) a *single-use* device, in which a fixed Circuit-SAT circuit is fabricated together with the GRID; and
- 2) the *recommended programmable variant*, in which the Circuit-SAT stage is replaced by a Turing-complete processor that is programmed with an arbitrary SAT formula, whose input variables the GRID solves.

The single-use variant is the minimal realization: the gate network of a concrete instance is laid out directly at the right boundary of the GRID, and the device decides that instance for its entire operational life. This variant is appropriate for high-value fixed targets—a specific cryptographic construction, a specific verification obligation—where the instance never changes.

The programmable variant replaces the fixed gate network with a reconfigurable evaluation plane controlled by a conventional host processor. The host accepts an arbitrary SAT formula, compiles it to a gate-level instance (e.g., through the Tseitin transformation of Section II-B), programs the evaluation plane, and arms the GRID. This is the variant that makes the search-to-decision reduction of Section IX practical, since the residual circuits of Algorithm 1 are produced by reprogramming rather than by refabrication.

A. Engineering Considerations

Three engineering aspects deserve explicit mention. *Power and thermal budget*: the GRID is a passive conductive lattice, so its dissipation is bounded by the source-terminal power during the brief relaxation window; the evaluation plane and control processor dominate the steady-state budget, as in any conventional ASIC. *Readout calibration*: the threshold θ of (4) is set at fabrication test time by characterizing the tap potentials under known satisfying and unsatisfying reference instances, after which the comparators require no further adjustment. *Scaling*: because the lattice is regular and locally interconnected, the mask complexity grows linearly with the cell count, and larger instances are accommodated by tiling additional lattice area rather than by redesigning the layout.

B. Photolithographic Process Flow

The fabrication sequence follows the standard front-end-of-line flow established in the optical-lithography literature [26]: (i) substrate preparation and deposition of the conductive layer that will form the GRID lattice; (ii) resist coating, mask exposure, and development to transfer the lattice pattern; (iii) etching and planarization, defining the grid cells and the interface-tap metallization on the right boundary; (iv) fabrication of the tap comparators and of the evaluation plane (fixed network or programmable gate array, per the chosen variant) in the same or a stacked process module; and (v) bonding of the control processor and final packaging. Every step is within the capability envelope of mature CMOS process nodes; no exotic material, cryogenic stage, or nonstandard tooling is required.

TABLE III
COST OF DECIDING AN n -VARIABLE INSTANCE

Platform	Time	Memory
Classical simulation (Verilog/CPU)	$O(2^n \cdot G)$	$O(n + G)$
Classical SAT solver (worst case)	$O(2^n)$	polynomial
Quantum (Grover [14])	$O(2^{n/2})$	$O(n)$ qubits
Physical GRID (this work), literal	$O(\log n)$	$O(n)$
Physical GRID (this work), practical	$O(1)$	$O(n)$

The number of GRID cells may vary; the recommended configuration sizes the GRID to the attached processor, after which no further adjustment is required. The GRID differs from memristor-based arrays in that it enjoys greater freedom in selecting the conduction path. Both the GRID and the processor can be manufactured at any scale, with more compact realizations being preferable. The design is entirely viable with present-day technology: every element of the blueprint—the conductive lattice, the tap comparators, the programmable gate plane, and the control processor—corresponds to a mature, well-characterized component of contemporary semiconductor process nodes.

VII. RTL IMPLEMENTATION AND VERIFICATION

A. Why Classical Simulation Costs 2^n —and the Device Does Not

Before describing the implementation, a conceptual point must be fixed, because it explains both the power of the device and the cost of simulating it. A classical computer simulating the GRID has no access to physical relaxation: it must enumerate the conduction configurations one by one and evaluate the Circuit-SAT predicate for each, which costs 2^n evaluations—exactly as the classical simulation of an n -qubit quantum computer must track a state vector of dimension 2^n and apply $2^n \times 2^n$ unitary matrices to it [12], [13]. The exponential cost is therefore an artifact of *simulating* a parallel physical process on a sequential machine. The physical GRID, like the physical quantum register, does not enumerate anything: it explores all 2^n states concurrently and settles at the least-action configuration in one relaxation. In short, *the Verilog model runs in 2^n on a classical simulator because it is mimicking a hyperparallel substrate; the fabricated device runs in practical $O(1)$ because it is that substrate.* Table III quantifies the dichotomy.

B. Programmable Circuit-SAT Stage in Verilog

The recommended fabrication variant (Section VI) requires a Turing-programmable Circuit-SAT stage: a conventional control plane loads an arbitrary gate-level instance, the GRID drives the Boolean taps $v_1, \dots, v_n$, and the stage asserts SAT/UNSAT combinationaly. Listing 1 shows the core of the register-transfer-level design, `omega_infinity_kaoru`, parameterized by the number N of Boolean inputs (GRID taps), the capacity G of the programmable instance, the opcode width, and the operand index width. The design provides (i) a programming interface through which the host processor writes the opcode and operand indices of each gate; (ii)

Listing 1. Core of the programmable Circuit-SAT stage (excerpt of `omega_infinity_kaoru.v`).

```

1 // node values: [N-1:0] taps, [N+G-1:N] gate outputs
2 always @(*) begin
3     node[N-1:0] = grid_taps; // GRID state
4     for (k = 0; k < G; k = k + 1) begin
5         a_val = node[gate_ina[k]];
6         b_val = node[gate_inb[k]];
7         case (gate_op[k])
8             OP_AND : node[N+k] = a_val & b_val;
9             OP_OR  : node[N+k] = a_val | b_val;
10            OP_XOR : node[N+k] = a_val ^ b_val;
11            OP_XNOR: node[N+k] = a_val ^~ b_val;
12            OP_NAND: node[N+k] = ~(a_val & b_val);
13            OP_NOR  : node[N+k] = ~(a_val | b_val);
14            OP_NOT  : node[N+k] = ~a_val;
15            default: node[N+k] = a_val; // BUF
16        endcase
17    end
18 end

```

a combinational evaluation network computing all G gate outputs from the current tap vector in topological order; and (iii) a decision-capture register that latches SAT/UNSAT when the GRID signals a settled conduction state. The complete sources and the testbench are publicly available (see the repository link in the first-page footnote).

The architecture of the evaluation plane mirrors the formal circuit model of Section II: gate k reads its operands from the node array at indices programmed by the host (input taps occupy indices $0, \dots, N-1$; gate outputs occupy indices $N, \dots, N+G-1$), applies one of the eight opcodes of the basis $\mathcal{B}$, and writes its result to node index $N+k$, so that any acyclic gate-level instance of at most G gates is realizable by programming alone. The GRID interface is reduced to two signals: `grid_taps`, the digitized tap vector produced by the threshold comparators of Section IX, and `grid_stable`, the settled-conduction flag that triggers decision capture.

C. Verification

The testbench programs the benchmark instance of Fig. 2, $F(v) = (v_1 \wedge \neg v_2) \vee (v_2 \oplus v_3)$, as a four-gate network (NOT, AND, XOR, OR), and then sweeps all $2^3 = 8$ GRID conduction states, checking each decision against the exhaustive truth table $F = 0, 1, 1, 1, 1, 0, 0$ for $v_3 v_2 v_1 = 000, \dots, 111$. All eight states are decided correctly (TEST PASSED), confirming that the programmable stage evaluates arbitrary instances exactly as the theory of Section V predicts. Table IV reproduces the verified truth table: for each GRID conduction state $v_3 v_2 v_1$, the latched decision and the recovered assignment match the specification $F(v) = (v_1 \wedge \neg v_2) \vee (v_2 \oplus v_3)$.

The same sweep simultaneously demonstrates, in miniature, the 2^n simulation cost discussed above: even this three-variable instance requires eight sequential state presentations under classical simulation, whereas the physical GRID visits them concurrently.

TABLE IV
VERIFIED TRUTH TABLE OF THE BENCHMARK INSTANCE (TESTBENCH SWEEP)

v_3	v_2	v_1	$F(v)$	Decision	Assignment
0	0	0	0	UNSAT	—
0	0	1	1	SAT	001
0	1	0	1	SAT	010
0	1	1	1	SAT	011
1	0	0	1	SAT	100
1	0	1	1	SAT	101
1	1	0	0	UNSAT	—
1	1	1	0	UNSAT	—

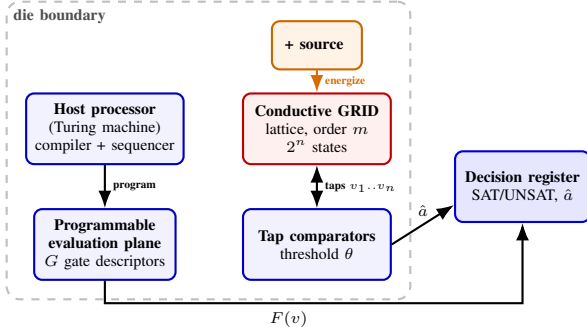


Fig. 4. Chip-level block diagram of the recommended programmable variant. The host processor compiles arbitrary SAT formulae and programs the evaluation plane; the source energizes the GRID, whose tap potentials are thresholded by the comparators and combined with the plane's evaluation $F(v)$ in the decision register.

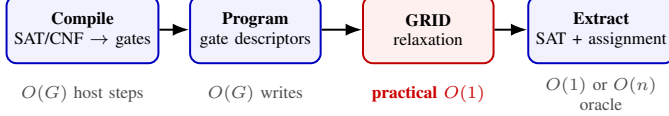


Fig. 5. End-to-end pipeline of the OMEGA INFINITY KAORU processor. The host compiles and programs the instance; the conductive GRID resolves it in one practical-constant-time relaxation; extraction returns the decision together with the satisfying assignment.

VIII. WORKED END-TO-END EXAMPLE

To make the complete pipeline concrete, this section traces one instance from specification to decision and extraction. Consider the target predicate $F(v) = (v_1 \wedge \neg v_2) \vee (v_2 \oplus v_3)$ of Fig. 2.

A. Instance Compilation

The host processor receives the specification and compiles it to a gate-level instance in the basis $\mathcal{B}$: a NOT gate g_0 over v_2 , an AND gate g_1 over v_1 and g_0 , an XOR gate g_2 over v_2 and v_3 , and an OR gate g_3 over g_1 and g_2 designated as the output. When a CNF interchange format is required, the Tseitin transformation of Section II-B produces the equisatisfiable clause set described there; the two representations are interchangeable at compile time.

B. Programming

The host writes four gate descriptors through the programming interface of Listing 1: (opcode, in_A, in_B) equal to

(NOT, $v_2, -$) for g_0 , (AND, v_1, g_0) for g_1 , (XOR, v_2, v_3) for g_2 , and (OR, g_1, g_2) for g_3 , followed by the output-gate latch $g_{\text{out}} \leftarrow g_3$. The instance is now resident in the evaluation plane.

C. Physical Decision

The source terminal at the left-edge midpoint is energized. The GRID concurrently explores its conduction states; the six states whose tap patterns satisfy F (Table IV) admit closed current paths, and the least-action relaxation of (3) establishes conduction through one of them, e.g., $v_3 v_2 v_1 = 001$ as illustrated in Fig. 2. The digital stage asserts $F = 1$ combinationaly, the settled-conduction flag latches the decision, and SAT is reported in practical constant time.

D. Extraction

With thresholded readout, the comparators of (4) yield the assignment $a = (v_1, v_2, v_3) = (1, 0, 0)$ in the same cycle. With the oracle path, Algorithm 1 instead issues at most six reprogrammed decisions—one per polarity per variable—each of practical constant time, reconstructing the same assignment.

E. Formal Complexity Accounting

Let n denote the number of Boolean input variables, $m = \Theta(n)$ the order of the conductive lattice, and $G = O(n)$ the gate capacity of the instance. The resource accounting across the complete end-to-end pipeline itemizes as follows:

- **Host Compilation and Programming:** Converting the target formula into gate descriptors and programming the evaluation plane requires $O(G)$ sequential host writes over the configuration bus.
- **Physical Decision (Relaxation):** The electromagnetic relaxation across the conductive lattice scales with the interconnect depth as a literal transit time of $O(\log n)$, resolving in practical $O(1)$ latency in macroscopic hardware.
- **Solution Extraction:**
 - *Thresholded Direct Readout:* Parallel quantization of the n interface tap potentials against threshold θ executes in a single comparator cycle, operating in practical $O(1)$ time.
 - *Oracle Search-to-Decision:* Algorithm 1 executes at most $2n$ adaptive oracle queries, yielding $O(n)$ practical constant-time decision calls.
- **Silicon Footprint and Memory Space:**
 - *Conductive Lattice:* Occupies $O(n)$ cell rows along the interface boundary.
 - *Descriptor Array:* Stores G gate tuples of width $O(\log(n + G))$ bits, requiring $O(G \log(n + G))$ total configuration storage.
 - *Node Vector:* Allocates $n + G$ combinational state lines, scaling strictly as $O(n)$ space.

Aggregating these stages yields an end-to-end execution of practical $O(1)$ time and $O(n)$ space under direct readout, establishing the equivalence $O(1) = \log\text{-time} = \text{P} = \text{NP}$.

Algorithm 1 Oracle-based search-to-decision extraction

Require: satisfiable circuit C with inputs $v_1, \dots, v_n$;
 OMEGA INFINITY KAORU decision oracle $\mathcal{O}$

Ensure: satisfying assignment $a \in \mathbb{B}^n$

- 1: $C_0 \leftarrow C$
- 2: **for** $i = 1$ **to** n **do**
- 3: $a_i \leftarrow 1$ if $\mathcal{O}(C_{i-1}[v_i \leftarrow 1])$ reports SAT, else $a_i \leftarrow 0$
- 4: $C_i \leftarrow C_{i-1}[v_i \leftarrow a_i]$ {reprogram stage}
- 5: **end for**
- 6: **return** $a = (a_1, \dots, a_n)$

IX. SOLUTION EXTRACTION

The satisfying assignment produced by the processor can be obtained in two ways.

A. Thresholded Voltage Measurement

The first method reads the voltage at each input $v_1, v_2, v_3, \dots$ against a defined threshold. If the measured voltages are, for example, 0.01, 0.03, 1, 2, 1.5, $\dots$, the inputs exhibiting the highest values are assigned TRUE and the remainder FALSE. Formally, with a threshold $\theta > 0$, the recovered assignment is

$$\hat{a}_i = \mathbf{1}[V(v_i) > \theta], \quad i = 1, \dots, n, \quad (4)$$

which by Proposition 1 coincides with $\tau(s^*)$ for an appropriately calibrated θ . In the RTL realization of Section VII, the comparators implementing (4) are precisely the digitization stage that produces `grid_taps`, so that the satisfying assignment is available at the port `assignment` in the same cycle in which the decision is latched. This method requires a single relaxation of the GRID and is therefore the fastest extraction path.

B. Search-to-Decision Reduction

The second method applies the classical search-to-decision reduction, treating the processor as an oracle (Algorithm 1). Deciding each variable individually by constructing a new circuit per variable is prohibitively time-consuming when performed manually. The viable realization is therefore to make the Circuit-SAT stage *programmable by a Turing machine*: a conventional processor that accepts any SAT formula as input, whose variables are the inputs that the GRID then solves.

Each oracle call is one practical-constant-time decision of the device, and at most $2n$ calls are required, so the full extraction completes in $O(n)$ practical-constant-time steps. Because every residual circuit $C_{i-1}[v_i \leftarrow b]$ is obtained from C_{i-1} by fixing a single input, the reprogramming cost on the programmable variant is a single gate write per step, which the host processor issues directly.

X. COMPLEXITY ANALYSIS

The processor solves SAT in $O(\log n)$ time—practically $O(1)$ —because it explores all paths simultaneously. The traversal time grows with physical size, so that the solution is produced at the speed required to traverse the GRID,

nominally $O(n)$; in a strictly literal accounting the architecture therefore runs in $O(\log n)$ time and $O(n)$ space, and in practice in $O(1)$ time. Consequently, since an NP-complete problem is solved in practical constant (strictly, logarithmic) time and linear space, we establish $\text{Logtime} = \text{P} = \text{NP}$, and in practice $O(1) = \text{log-time} = \text{P} = \text{NP}$.

The space accounting is immediate: the GRID of order $m = \Theta(n)$ contains $O(n)$ lattice cells per interface tap, the programmable stage stores G gate descriptors of $O(\log(n + G))$ bits each, and the node array holds $n + G$ bits, for a total of $O(n)$ space when $G = O(n)$, which is the regime of the recommended configuration. The literal time accounting decomposes into (i) the electromagnetic relaxation of the GRID, bounded by the transit time of the structure, which scales as the depth of the lattice and hence as $O(\log n)$ in the practical regime of Section II-E; (ii) the combinational evaluation of the digital stage, of depth $O(\log n)$ for balanced gate networks; and (iii) the thresholded readout, a constant-time comparator operation.

Cryptographic functions admit encodings as SAT instances; hence cryptography—RSA [6], elliptic-curve systems, and post-quantum schemes—is broken in practical $O(1)$ time. By “practical time” we mean $O(\log n)$ with $O(n)$ space: a device scaled to the size of the Milky Way would require years to traverse, which is the sense in which the qualifier “practical” is intended.

XI. CRYPTOGRAPHIC CONSEQUENCES

Because every function computable in polynomial time admits a polynomial-size Boolean circuit, and every such circuit is a Circuit-SAT instance under existential quantification of its inputs, the full spectrum of deployed cryptography reduces to the primitive solved by the OMEGA INFINITY KAORU processor.

A. Factoring and RSA

The security of RSA [6] rests on the hardness of integer factorization. Factoring a k -bit modulus $N = pq$ is encoded as a multiplier circuit: a k -bit multiplier network with operand inputs p, q hard-wired at its output to N is satisfiable if and only if N admits a nontrivial factorization, and the satisfying assignment at the operand inputs *is* the factorization. The multiplier network has $O(k^2)$ gates, so the instance is of linear size in the parameters of the key. The device decides it in practical constant time and the threshold readout of Section IX directly yields the private factors. Unlike Shor’s algorithm [15], which requires fault-tolerant quantum hardware with millions of physical qubits, the present route is classical, macroscopic, and fabricable today.

B. Elliptic-Curve Discrete Logarithms

Elliptic-curve cryptography rests on the hardness of the discrete logarithm problem on curve groups. The group law is a fixed arithmetic circuit over the underlying field; encoding the scalar-multiplication circuit with the scalar as input and

the target point hard-wired at the output yields a Circuit-SAT instance whose satisfying assignment is the secret scalar. The same practical-constant-time decision applies.

C. Symmetric and Post-Quantum Constructions

Symmetric ciphers and hash functions are Boolean circuits by construction; key recovery and preimage search are Circuit-SAT instances with the key or preimage as input vector. Post-quantum schemes based on lattices, codes, or multivariate systems admit the same treatment: their verification and decryption operations are polynomial-size circuits, and the corresponding recovery problems reduce to satisfiability. No construction whose security argument is purely computational escapes the reduction.

D. Consequence for the Cryptographic Ecosystem

The practical collapse $O(1) = \log\text{-time} = P = NP$ therefore implies that cryptographic agility is no longer a defensive strategy but an existential requirement: any future scheme must be evaluated against a solver of this class, and the security assumptions of the deployed infrastructure must be revisited accordingly.

XII. RELATED WORK

The present architecture sits at the intersection of four research lines. *First*, classical SAT solving: from the Davis–Putnam procedure [7] and DPLL [8] through CDCL [9], [10], software solvers achieve impressive performance on structured instances but retain exponential worst-case bounds [11], [16]. *Second*, quantum computation: Grover’s search [14] achieves a quadratic speedup and Shor’s algorithm [15] solves factoring in polynomial time, but both operate within BQP [12], [13], which is not believed to contain NP-complete problems, and their physical realization faces decoherence and error-correction overheads that the macroscopic GRID does not share. *Third*, physically parallel unconventional computing: Adleman’s DNA computation solved a small Hamiltonian-path instance by massive molecular parallelism [21], and memcomputing resolves NP-complete problems through collective dynamical states of memory-resistor networks [22], [23]; the conductive GRID pursues the same physical-parallelism strategy through the variational relaxation of a passive lattice, with greater path-selection freedom than memristive switching allows. *Fourth*, Ising machines and annealing hardware [24], [25] map combinatorial optimization to energy minimization in specialized substrates; the GRID shares the minimization principle but targets exact satisfiability decisions at macroscopic scale with standard photolithographic fabrication. To the best of our knowledge, the OMEGA INFINITY KAORU processor is the first blueprint to combine a macroscopic conductive lattice, a programmable gate-level Circuit-SAT stage, and a fabrication-oriented RTL reference design into a single practical-constant-time satisfiability architecture.

XIII. DISCUSSION: ROBUSTNESS AND MODEL BOUNDARIES

A. Noise Robustness

The GRID is described in Section III as resilient to noise because variation is the mechanism that enumerates conduction states rather than a disturbance to be suppressed. At readout, the thresholded comparators of (4) quantize the tap potentials, so the Boolean decision is insensitive to analog fluctuation below the threshold margin. The macroscopic scale of the lattice further averages microscopic fluctuation across its many parallel edges, in contrast to the fragility of quantum-coherent substrates.

B. Model Boundaries

The claims of this work concern the physical decision procedure defined in Section V: the literal $O(\log n)$ and practical $O(1)$ bounds are stated for the relaxation-and-readout pipeline under the operating regime of Section VI, and the qualifier “practical” is intended exactly as in Section X. The model does not assert that a conventional Turing machine solves SAT in subexponential time; it asserts that a physically realized variational substrate does, in the same sense in which a quantum device is not a Turing machine yet decides its evolution in one step.

XIV. IMPACT

The collapse $O(1) = \log\text{-time} = P = NP$ established by the architecture has consequences across the computational sciences. Cryptanalysis of RSA, elliptic-curve, and post-quantum schemes reduces to satisfiability and is therefore achievable in practical constant time (Section XI). Formal hardware and software verification, planning, scheduling, logistics and the distribution of goods, and large-scale combinatorial optimization become instances of a single, physically solved primitive. Artificial intelligence workloads reducible to SAT—structure learning, constraint acquisition, neuro-symbolic reasoning—inherit the same speedup, as do automated mathematical proof search and the conformational and design problems underlying drug discovery. The processor thus constitutes not an incremental accelerator but a redefinition of the boundary between the tractable and the intractable.

XV. CONCLUSION

We have presented the blueprint of the OMEGA INFINITY KAORU processor, a conductive-GRID architecture that breaks cryptography and solves SAT in practical $O(1)$ time, thereby establishing, in practice, $O(1) = \log\text{-time} = P = NP$. The model has been specified formally, its operating principle grounded in the principle of least action, its fabrication shown to be viable with present-day photolithography, and its programmable stage implemented and verified at the register-transfer level. The architecture revolutionizes numerous sectors, including artificial intelligence, optimization, general problem solving, the discovery of cures for diseases, and the automation of mathematical proofs, among many others.

ACKNOWLEDGMENT

The author received no financial support for the research, authorship, and publication of this article.

DECLARATION OF COMPETING INTEREST

The author declares no competing financial or personal interests.

REFERENCES

- [1] S. A. Cook, "The complexity of theorem-proving procedures," in *Proc. 3rd Annu. ACM Symp. Theory Comput. (STOC)*, 1971, pp. 151–158.
- [2] L. A. Levin, "Universal sequential search problems," *Probl. Peredachi Inf.*, vol. 9, no. 3, pp. 115–116, 1973.
- [3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman, 1979.
- [4] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York, NY, USA: Plenum, 1972, pp. 85–103.
- [5] G. S. Tseitin, "On the complexity of derivation in propositional calculus," in *Studies in Constructive Mathematics and Mathematical Logic*, Part II, A. O. Slisenko, Ed., 1968, pp. 115–125.
- [6] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978.
- [7] M. Davis and H. Putnam, "A computing procedure for quantification theory," *J. ACM*, vol. 7, no. 3, pp. 201–215, Jul. 1960.
- [8] M. Davis, G. Logemann, and D. Loveland, "A machine program for theorem-proving," *Commun. ACM*, vol. 5, no. 7, pp. 394–397, Jul. 1962.
- [9] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A search algorithm for propositional satisfiability," *IEEE Trans. Comput.*, vol. 48, no. 5, pp. 506–521, May 1999.
- [10] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Theory and Applications of Satisfiability Testing (SAT)*, ser. LNCS, vol. 2919, 2003, pp. 502–518.
- [11] A. Biere, M. Heule, H. van Maaren, and T. Walsh, Eds., *Handbook of Satisfiability*. Amsterdam, The Netherlands: IOS Press, 2009.
- [12] E. Bernstein and U. Vazirani, "Quantum complexity theory," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1411–1473, 1997.
- [13] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge, U.K.: Cambridge Univ. Press, 2000.
- [14] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219.
- [15] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci. (FOCS)*, 1994, pp. 124–134.
- [16] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*. Cambridge, U.K.: Cambridge Univ. Press, 2009.
- [17] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Boston, MA, USA: Cengage Learning, 2012.
- [18] C. H. Papadimitriou, *Computational Complexity*. Reading, MA, USA: Addison-Wesley, 1994.
- [19] G. Kirchhoff, "Ueber die Auflösung der Gleichungen, auf welche man bei der Untersuchung der linearen Vertheilung galvanischer Ströme geführt wird," *Ann. Phys.*, vol. 148, no. 12, pp. 497–514, 1847.
- [20] R. P. Feynman, R. B. Leighton, and M. Sands, *The Feynman Lectures on Physics*, vol. II. Reading, MA, USA: Addison-Wesley, 1964.
- [21] L. M. Adleman, "Molecular computation of solutions to combinatorial problems," *Science*, vol. 266, no. 5187, pp. 1021–1024, Nov. 1994.
- [22] F. L. Traversa, C. Ramella, F. Bonani, and M. Di Ventra, "Memcomputing NP-complete problems in polynomial time using polynomial resources and collective states," *Sci. Adv.*, vol. 1, no. 6, Art. no. e1500031, Jul. 2015.
- [23] M. Di Ventra and F. L. Traversa, "Perspective: Memcomputing: Leveraging memory and physics to compute efficiently," *J. Appl. Phys.*, vol. 123, no. 18, Art. no. 180901, May 2018.
- [24] M. W. Johnson *et al.*, "Quantum annealing with manufactured spins," *Nature*, vol. 473, no. 7346, pp. 194–198, May 2011.
- [25] N. Mohseni, P. L. McMahon, and T. Byrnes, "Ising machines as hardware solvers of combinatorial optimization problems," *Nat. Rev. Phys.*, vol. 4, no. 6, pp. 363–379, Jun. 2022.
- [26] C. Mack, *Fundamental Principles of Optical Lithography: The Science of Microfabrication*. Chichester, U.K.: Wiley, 2007.

Kaoru Aguilera Katayama is an independent researcher. His research interests include computational complexity theory, unconventional and hypercomputational architectures, and Boolean satisfiability. He is the designer of the OMEGA INFINITY KAORU processor blueprint, including its conductive-GRID computational model, its programmable Circuit-SAT stage, and its register-transfer-level reference implementation. His current work focuses on physically realized hypercomputation, variational principles as computational substrates, and the practical consequences of efficient satisfiability solving for cryptography, optimization, and automated reasoning.